AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2026

MS-SWIFT: Fine-tuning Mô hình Ngôn ngữ Lớn

Nguyễn Phúc Thịnh Đinh Quang Vinh
Content Revision: Nguyễn Thế Hào

I. Giới thiệu

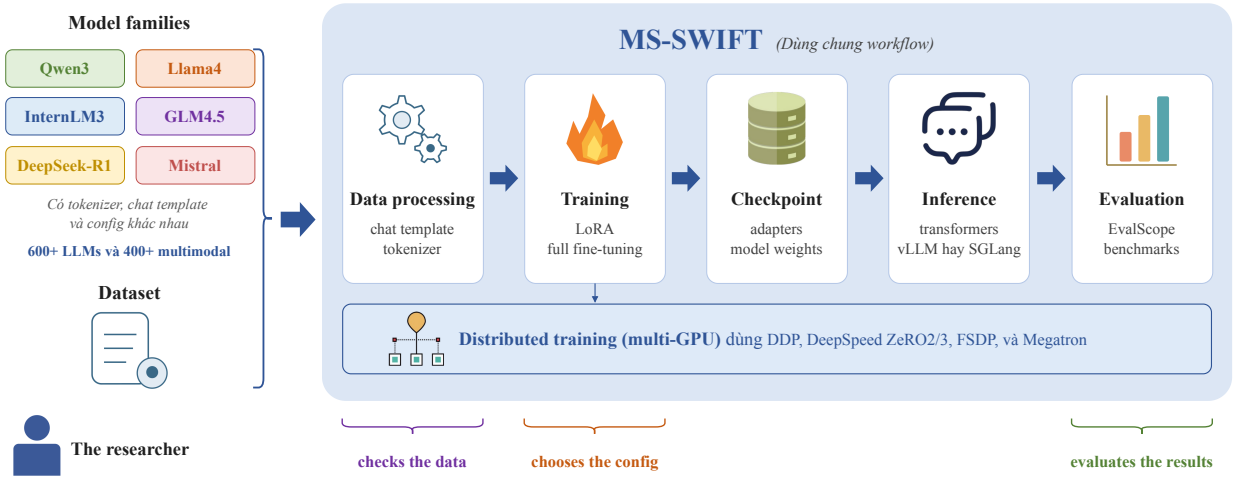
Large Language Models (LLMs), hay mô hình ngôn ngữ lớn, đã mở ra nhiều cơ hội cho nghiên cứu cũng như ứng dụng AI. Ngày càng nhiều công ty công khai trọng số model, chẳng hạn Qwen, Llama và InternLM. Nhờ đó, người học có thể tự tải model về để tìm hiểu và thử nghiệm. Ta được chủ động chọn model và thử áp dụng nó vào bài toán cụ thể của mình, thay vì chỉ dùng các dịch vụ có sẵn.

Tuy vậy, có model trong tay mới chỉ là điểm bắt đầu. Muốn model thật sự hợp với dữ liệu và mục tiêu của mình, tải model về rồi chạy một lệnh là chưa đủ. Huấn luyện thực ra chỉ là một khâu trong cả quy trình. Trước khi train, ta phải chọn model và tokenizer, rồi chuẩn bị dataset đúng định dạng model cần. Ta cũng phải chọn phương pháp fine-tuning hợp với mục tiêu và tài nguyên đang có. Train xong, ta kiểm tra log và checkpoint, rồi mới đem model đi inference và đánh giá. Các khâu này nối tiếp nhau, nên sai ở một bước thì các bước sau cũng bị ảnh hưởng.

Với một model và dataset nhỏ, ta hoàn toàn có thể tự dựng pipeline bằng những thư viện quen thuộc như `PyTorch` và `Transformers`. Nhưng chỉ cần đổi model, đổi dataset hay chuyển từ một GPU sang nhiều GPU, cấu hình và định dạng dữ liệu cũng phải đổi theo. Nếu thực nghiệm nào cũng phải viết lại và kiểm tra từng thành phần, phần lớn công sức sẽ dồn vào bảo trì code thay vì mục tiêu ban đầu. Vậy có cách nào gom các bước trên vào một workflow thống nhất mà vẫn tự do chọn model, dữ liệu và phương pháp huấn luyện không?

MS-SWIFT (Scalable lightWeight Infrastructure for Fine-Tuning) của ModelScope ra đời để giải quyết đúng vấn đề này. Thay vì ghép nhiều thư viện lại với nhau, MS-SWIFT đưa mọi khâu vào chung một bộ lệnh `swift`. Bộ lệnh này đi từ chuẩn bị dữ liệu, huấn luyện, inference, đánh giá cho tới export model. Muốn đổi model, dataset hay phương pháp fine-tuning, ta thường chỉ cần đổi vài tham số dòng lệnh chứ không phải viết lại pipeline. Có thể bạn sẽ hỏi, sao không dùng luôn những thư viện quen thuộc hơn như `vLLM` hay `Unsloth`? Thật ra mỗi thư viện này chỉ lo một phần của quy trình. `vLLM` là engine để inference và serving model với tốc độ cao, nên nó không phục vụ việc huấn luyện. `Unsloth` thì tập trung tăng tốc và giảm bộ nhớ khi fine-tune, chứ không lo quản lý cả workflow. MS-SWIFT không cạnh tranh với các công cụ đó mà dùng chúng như những thành phần bên trong. Chẳng hạn, khi inference, ta có thể chọn `vLLM` làm backend chỉ bằng tham số `-infer_backend vllm`. Như vậy ta vẫn tận dụng được điểm mạnh

của từng thư viện, mà chỉ phải học và duy trì một workflow duy nhất. Hình 1 cho thấy tổng quan các khâu mà MS-SWIFT hỗ trợ.



Hình 1: Vai trò của MS-SWIFT trong quy trình fine-tuning mô hình ngôn ngữ lớn.

Các phần tiếp theo sẽ hướng dẫn bạn dùng workflow này từng bước. Đầu tiên, ta tìm hiểu MS-SWIFT, cài đặt môi trường và chọn model. Kế đến, ta tải dataset, xem qua cấu trúc ban đầu rồi chuyển dữ liệu sang định dạng phù hợp. Sau đó, bài đọc trình bày cách cấu hình và chạy fine-tuning, cách theo dõi quá trình huấn luyện và lưu checkpoint. Cuối cùng, ta dùng model đã huấn luyện để inference và evaluation. Những khái niệm như chat template, LoRA, full fine-tuning và distributed training sẽ được giải thích ở phần liên quan.

Mục lục

I. Giới thiệu **1**

II. Tổng quan MS-SWIFT **4**

III. Các thành phần trong quá trình fine-tuning **5**

III.1. Model và tokenizer 5

III.2. Dataset và chat template 5

III.3. Phương pháp fine-tuning 7

III.4. Cấu hình huấn luyện 7

III.5. GPU và huấn luyện phân tán 8

III.6. Log và checkpoint 9

IV. Đọc một câu lệnh fine-tuning **9**

V. Luồng xử lý của `swift sft` **10**

VI. Thực hành fine-tune Qwen2.5 trên GSM8K với MS-SWIFT **11**

VI.1. Cấu hình chung 11

VI.2. Chuẩn bị dữ liệu GSM8K theo định dạng `messages` 12

VI.3. Huấn luyện mô hình bằng `swift sft` kết hợp LoRA và DeepSpeed ZeRO-2 . 13

VI.4. Đọc kết quả 15

VI.5. Chạy thử với `swift infer` và gộp LoRA bằng `swift export` 16

VII. Thực hành chèn một khối mới vào giữa Qwen2.5 và huấn luyện bằng MS-SWIFT **17**

VII.1. Cấu hình và dữ liệu 17

VII.2. Ý tưởng: khối `mid_adapter` 18

VII.3. Một model Hugging Face gồm những gì? 18

VII.4. Viết kiến trúc mới 20

VII.5. Dựng model mới và kiểm tra trước khi train 21

VII.6. Đăng ký `model_type` mới cho MS-SWIFT 22

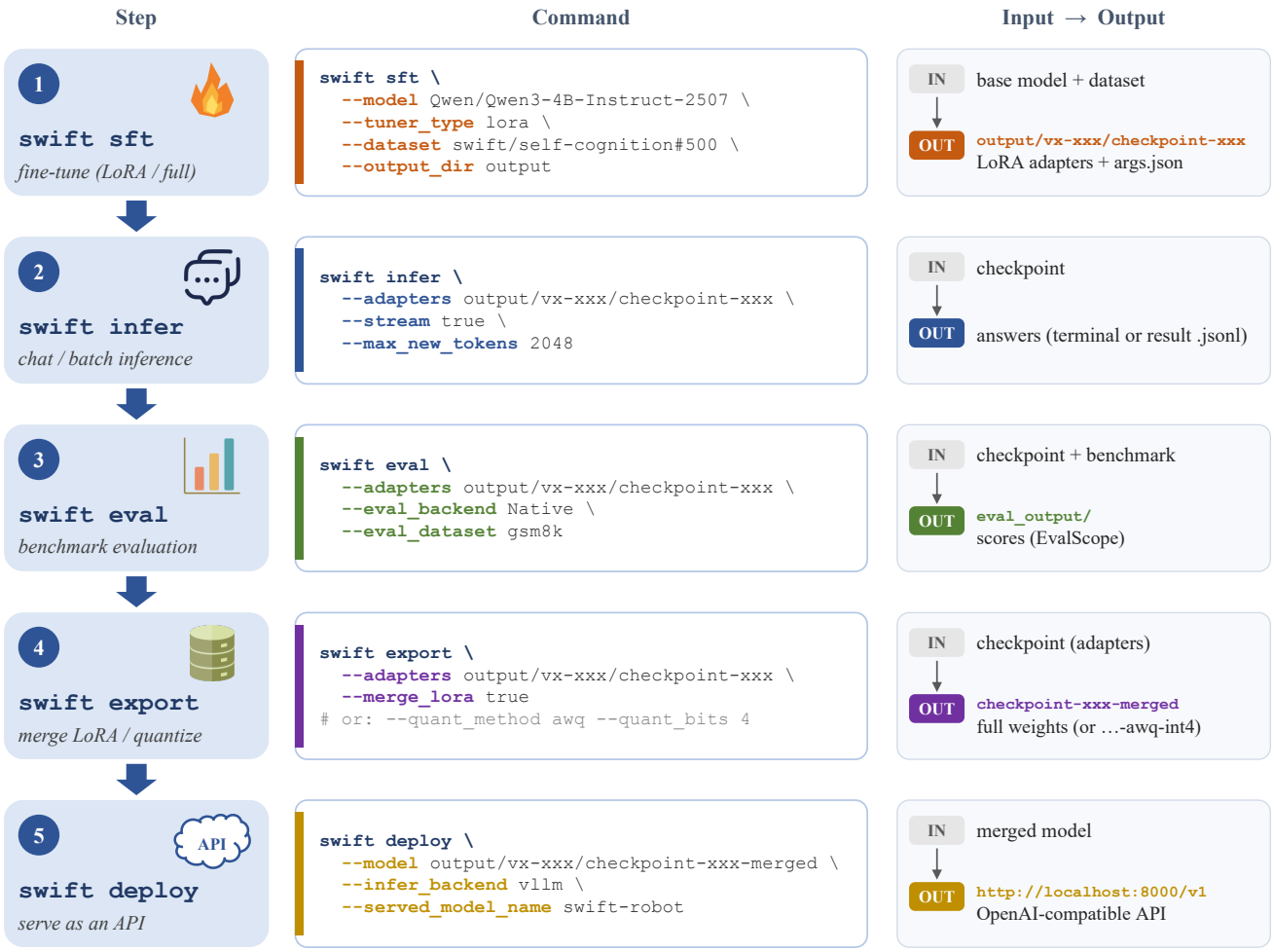
VII.7. Huấn luyện chỉ riêng khối mới 23

VII.8. Kiểm tra khối mới và suy luận của mô hình 24

Phụ lục **26**

II. Tổng quan MS-SWIFT

MS-SWIFT hỗ trợ các bước từ fine-tuning đến đánh giá và triển khai LLM qua các lệnh CLI (Command-line Interface). Để bắt đầu một thực nghiệm, ta chỉ cần chỉ định model, dataset và cấu hình huấn luyện. Hình 2 minh họa cách dùng năm lệnh chính trong một workflow fine-tuning với LoRA.



Hình 2: Ví dụ dùng các lệnh CLI của MS-SWIFT để fine-tune, chạy inference, đánh giá, export và triển khai mô hình.

Trong ví dụ này, `swift sft` fine-tune model trên dataset và lưu adapter LoRA vào checkpoint. Train xong, ta có thể dùng `swift infer` để thử câu trả lời của model, hoặc `swift eval` để đo kết quả trên benchmark. Khi cần triển khai, `swift export` gộp adapter vào trọng số model. Sau đó `swift deploy` đưa model lên dưới dạng API để các ứng dụng khác gọi tới.[1]

Bạn không bắt buộc phải chạy đủ các bước này, cũng không cần theo đúng thứ tự trong hình. Hãy chọn những bước phù hợp với mục tiêu thực nghiệm. Chẳng hạn, nếu chỉ muốn đánh giá kết quả fine-tuning, bạn có thể dừng sau bước `swift eval`.

[1]Tham khảo tài liệu chính thức của MS-SWIFT về quick start, evaluation và inference và deployment.

III. Các thành phần trong quá trình fine-tuning

Để thiết lập một lần fine-tuning, ta cần xác định model sẽ học, dữ liệu để học, cách cập nhật tham số và cách chạy trên phần cứng. Mỗi lựa chọn này ứng với một nhóm cấu hình dưới đây.

III.1. Model và tokenizer

Mọi lần fine-tuning đều bắt đầu từ một model nền. Ta chọn model bằng tham số `-model`, với giá trị là tên model hoặc đường dẫn tới trọng số đã lưu.

Đi cùng model là tokenizer, phần chuyển văn bản thành các token mà model xử lý được. Tokenizer phải khớp với model, vì mỗi dòng mô hình có thể dùng bộ từ vựng và token đặc biệt khác nhau.

Ví dụ, với câu đầu vào `Hello!`, tokenizer tách văn bản thành các token rồi ánh xạ chúng sang các ID tương ứng. Dãy ID này mới là đầu vào mà model thật sự dùng.

```python
from transformers import AutoTokenizer

model_id = "Qwen/Qwen2.5-1.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id)

text = "Hello!"
tokens = tokenizer.tokenize(text)
token_ids = tokenizer.encode(text, add_special_tokens=False)

print("Text:", text)
print("Tokens:", tokens)
print("Token IDs:", token_ids)
print("Decoded:", tokenizer.decode(token_ids))

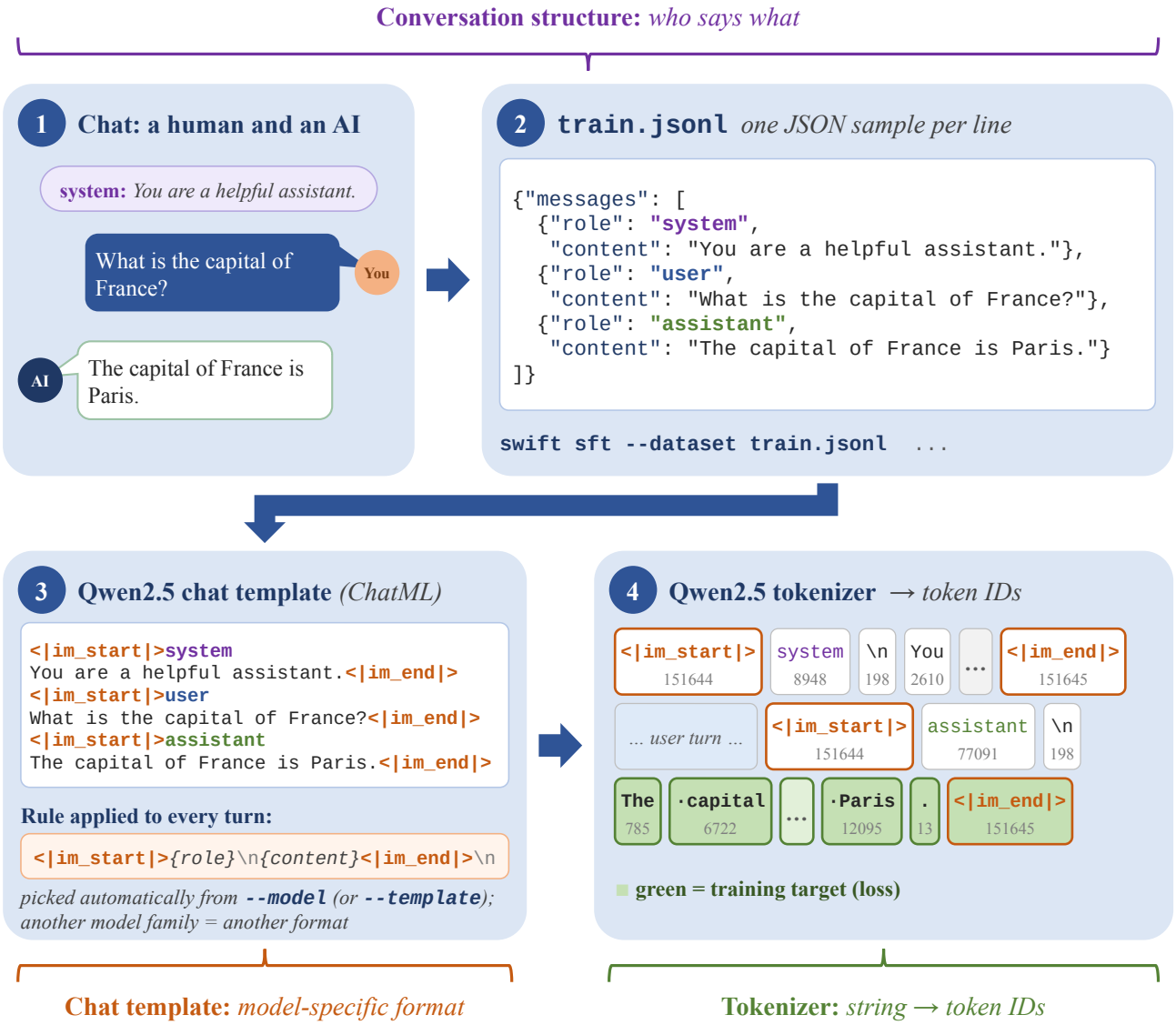
>>> Text: Hello!
>>> Tokens: ['Hello', '!']
>>> Token IDs: [9707, 0]
>>> Decoded: Hello!
```

Trong lệnh `swift sft`, tham số `-model Qwen/Qwen2.5-1.5B-Instruct` chọn model nền. MS-SWIFT tự nạp tokenizer tương ứng khi chuẩn bị đầu vào, nên bạn không cần viết lại bước tokenize như ví dụ minh họa trên.

III.2. Dataset và chat template

Với supervised fine-tuning (SFT), ta cho model xem thật nhiều ví dụ mẫu. Mỗi ví dụ cho model biết với một đầu vào cụ thể thì câu trả lời đúng trông ra sao. Với dữ liệu hội thoại, mỗi ví dụ giống một đoạn chat ngắn gồm nhiều lượt. Lượt `system` đặt chỉ dẫn chung cho model, còn lượt `user` là câu hỏi hay yêu cầu của người dùng. Lượt `assistant` là câu trả lời mà ta muốn model học theo.

Vậy một đoạn chat như thế đi qua những bước nào trước khi đến được model? Hình 3 đi theo một ví dụ cụ thể, từ đoạn hội thoại ban đầu cho tới dãy token ID.



Hình 3: Một đoạn hội thoại được chuyển thành dữ liệu JSONL, áp dụng chat template rồi tokenize với Qwen2.5.

Trong ví dụ, người dùng hỏi thủ đô của nước Pháp và model trả lời là Paris. Đoạn chat được lưu dưới dạng JSON trong trường `messages`. Mỗi lượt là một mục gồm hai trường, `role` cho biết ai đang nói và `content` chứa nội dung câu nói. Trong file JSONL, mỗi dòng chứa trọn một mẫu như vậy (hình chỉ ngắt dòng phần JSON cho dễ đọc). Khi huấn luyện, ta chỉ cần trỏ tới file này bằng tham số `-dataset train.jsonl`.

Tiếp theo, chat template ghép các lượt hội thoại thành một chuỗi theo định dạng riêng của model. Trong hình, template của Qwen2.5 dùng các token đặc biệt như `<|im_start|>` và `<|im_end|>` để đánh dấu ranh giới từng lượt. Nói cách khác, `messages` cho biết ai nói và nói gì. Chat template thì quyết định hội thoại được trình bày cho model theo cách nào.

Cuối cùng, tokenizer biến chuỗi đã định dạng thành các token ID. Theo cách tính loss trong hình, chỉ phần trả lời của `assistant` được dùng làm mục tiêu huấn luyện. Chỉ dẫn và câu hỏi chỉ đóng vai trò ngữ cảnh để model học cách trả lời.

III.3. Phương pháp fine-tuning

Có đầu vào rồi, câu hỏi tiếp theo là nên cập nhật bao nhiêu phần của model. Đây là chuyện đánh đổi giữa mức độ thay đổi model và lượng tài nguyên phải bỏ ra. Có hai lựa chọn thường gặp.

- **Full fine-tuning** cập nhật toàn bộ tham số của model. Model có thể thay đổi nhiều nhất, nhưng cái giá là tốn nhiều bộ nhớ lúc fine-tuning. Ngoài trọng số, GPU còn phải lưu gradient và trạng thái optimizer cho từng tham số.
- **LoRA** giữ nguyên trọng số gốc và chỉ huấn luyện thêm một lượng nhỏ trọng số mới. Số tham số cần học ít hơn nhiều, nên LoRA tiết kiệm đáng kể bộ nhớ và thời gian. Đổi lại, model chỉ được điều chỉnh trong phạm vi hẹp hơn.

Trong MS-SWIFT, ta chọn phương pháp bằng tham số `-tuner_type` (ví dụ `full` hoặc `lora`). Với LoRA, kết quả lưu lại thường chỉ là phần trọng số nhỏ được học thêm, gọi là adapter. Khi dùng, ta nạp adapter kèm model gốc hoặc gộp hẳn nó vào trọng số model.

III.4. Cấu hình huấn luyện

Có dữ liệu và phương pháp fine-tuning rồi, ta cần quyết định model sẽ học *như thế nào*. Học trong bao lâu, mỗi lần điều chỉnh nhiều hay ít, mỗi bước xem bao nhiêu dữ liệu? Năm tham số dưới đây là những thứ nên đọc trước tiên.

- `-num_train_epochs` là số lần model đi qua toàn bộ dataset. Học quá ít thì model chưa kịp nắm dữ liệu, học quá nhiều thì dễ học thuộc lòng (overfitting).
- `-learning_rate` là tốc độ học. Nó quyết định mỗi lần cập nhật thì tham số được chỉnh mạnh hay nhẹ.
- `-per_device_train_batch_size` là số mẫu mỗi GPU xử lý trong một lượt forward/backward.
- `-gradient_accumulation_steps` là số lượt forward/backward được cộng dồn gradient trước mỗi lần cập nhật tham số.
- `-max_length` là độ dài tối đa của chuỗi token. Mặc định `-truncation_strategy` là `delete`, nên mẫu dài hơn giới hạn này sẽ bị loại bỏ. Nếu đặt `left` hoặc `right`, mẫu sẽ bị cắt bớt thay vì bị loại.

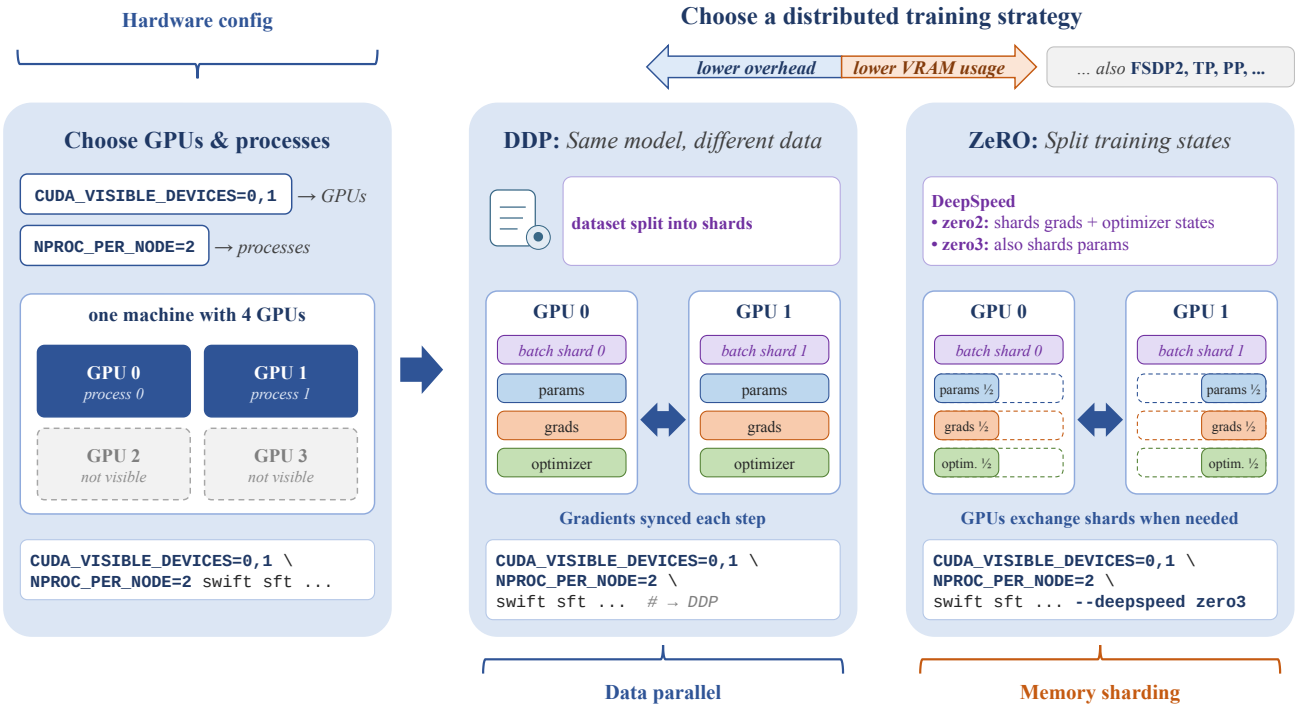
Các tham số này ảnh hưởng lẫn nhau, nên cần được cân nhắc cùng lúc. Tăng batch size hoặc `-max_length` thường làm tốn thêm bộ nhớ GPU. Khi GPU không đủ chỗ cho batch lớn hơn, ta có thể giữ batch nhỏ trên mỗi GPU và tích lũy gradient qua nhiều lượt. Số mẫu thực sự góp vào mỗi lần cập nhật được gọi là batch size hiệu dụng, và nó bằng

batch size hiệu dụng = `per_device_train_batch_size`×`gradient_accumulation_steps`×số GPU.

Giả sử mỗi GPU chạy batch size 2 và tích lũy 8 lượt, trên 2 GPU. Khi đó mỗi lần cập nhật tham số dựa trên $2 \times 8 \times 2 = 32$ mẫu.

III.5. GPU và huấn luyện phân tán

Cấu hình phần cứng quyết định dùng bao nhiêu GPU và chạy bao nhiêu tiến trình huấn luyện. Ở đây, một tiến trình (process) là một chương trình Python huấn luyện chạy độc lập, do `torchrun` khởi động. Thông thường mỗi tiến trình điều khiển một GPU. Khi chạy trên một máy, biến môi trường `CUDA_VISIBLE_DEVICES` chọn GPU, còn `NPROC_PER_NODE` đặt số tiến trình được khởi chạy trên máy đó.

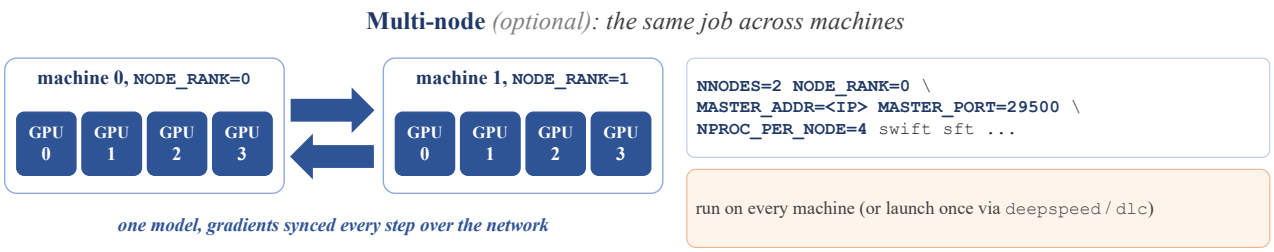


Hình 4: Chọn GPU và số tiến trình, rồi chọn một chiến lược huấn luyện phân tán. DDP chia dữ liệu, còn ZeRO chia bộ nhớ.

Với Distributed Data Parallel (DDP), mỗi GPU giữ một bản sao model và xử lý một phần dữ liệu. Các tiến trình đồng bộ gradient với nhau để model được cập nhật nhất quán. Như vậy DDP chia việc xử lý dữ liệu, nhưng không tự chia nhỏ trọng số model giữa các GPU.

Nếu model hoặc trạng thái huấn luyện không vừa bộ nhớ của một GPU, ta cần thêm cơ chế chia bộ nhớ, chẳng hạn ZeRO trong DeepSpeed. Chọn cấu hình phân tán nào là tùy vào nhu cầu của thực nghiệm và lượng VRAM bạn có.

Khi một máy không còn đủ GPU, ta có thể mở rộng việc huấn luyện ra nhiều máy. Cách này gọi là huấn luyện multi-node. Mỗi máy (node) vẫn chạy các tiến trình như khi chạy trên một máy. Điểm khác là tất cả cùng tham gia một job và đồng bộ gradient với nhau qua mạng sau mỗi bước. Vì vậy cuối cùng vẫn chỉ có một model được huấn luyện chung. Trong MS-SWIFT, `NNODES` khai báo tổng số máy và `NODE_RANK` cho biết thứ tự của từng máy. Hai biến `MASTER_ADDR` và `MASTER_PORT` chỉ tới máy điều phối, tức máy có `NODE_RANK=0`. Lệnh huấn luyện thường được chạy trên mọi máy, mỗi máy với `NODE_RANK` riêng của mình. Các máy phải trao đổi gradient liên tục, nên tốc độ mạng giữa chúng ảnh hưởng trực tiếp tới tốc độ huấn luyện.



Hình 5: Huấn luyện multi-node. Cùng một job chạy trên nhiều máy và đồng bộ gradient qua mạng ở mỗi bước.

III.6. Log và checkpoint

Trong lúc huấn luyện, MS-SWIFT ghi hai loại kết quả vào thư mục đặt bằng `-output_dir`. Loại thứ nhất là log, ghi lại tiến trình học như loss và số bước đã chạy, giúp ta theo dõi model đang học ra sao. Loại thứ hai là checkpoint, lưu model hoặc adapter tại những thời điểm đã cấu hình. Nhờ checkpoint, ta có thể nạp lại kết quả để inference và đánh giá. Dù vậy, training loss giảm vẫn chưa đủ để kết luận model đã tốt hơn. Có thể model chỉ đang học thuộc dữ liệu huấn luyện, nên kết quả cần được kiểm tra trên dữ liệu tách biệt với tập huấn luyện.

IV. Đọc một câu lệnh fine-tuning

Đến đây, ta đã biết từng thành phần riêng lẻ. Câu lệnh dưới đây ghép chúng lại với nhau. Đường dẫn dataset và các giá trị cấu hình chỉ là ví dụ để tập đọc lệnh. `swift sft` là lệnh supervised fine-tuning của MS-SWIFT, tức là dạy model trả lời theo các cặp câu hỏi và câu trả lời mẫu trong dataset. Các tham số phía sau cho lệnh này biết dùng model nào, dữ liệu nào và học theo cách nào.

```
NPROC_PER_NODE=2 \
CUDA_VISIBLE_DEVICES=0,1 \
swift sft \
    --model Qwen/Qwen2.5-1.5B-Instruct \
    --dataset /path/to/train.jsonl \
    --tuner_type lora \
    --num_train_epochs 1 \
    --per_device_train_batch_size 1 \
    --gradient_accumulation_steps 4 \
    --learning_rate 1e-4 \
    --max_length 1024 \
    --output_dir output
```

Ta có thể đọc câu lệnh này theo bốn nhóm, mỗi nhóm trả lời một câu hỏi.

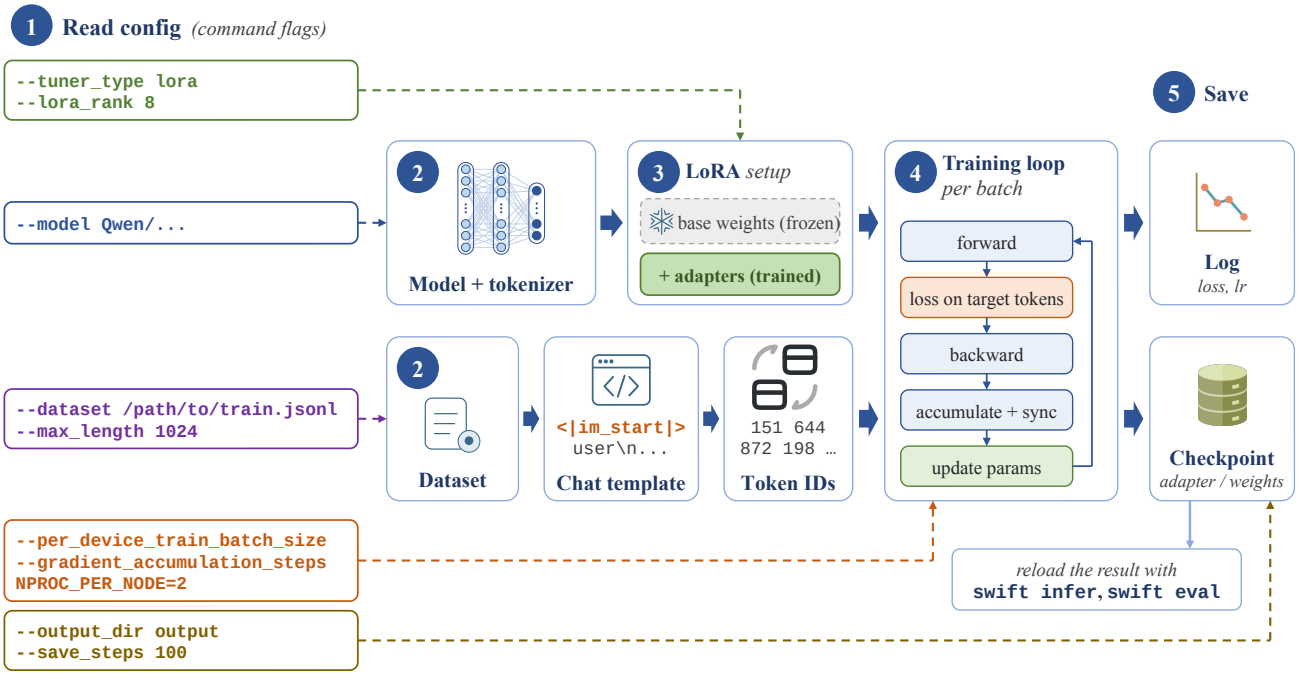
- **Chạy trên phần cứng nào?** Hai dòng đầu chọn GPU 0 và GPU 1, rồi khởi chạy hai tiến trình, mỗi tiến trình điều khiển một GPU.

- **Học model nào, từ dữ liệu nào?** `swift sft` chạy supervised fine-tuning cho model `Qwen2.5-1.5B-Instruct` với dữ liệu trong file `train.jsonl`.
- **Học theo cách nào?** `-tuner_type lora` chọn LoRA, nên chỉ một phần nhỏ trọng số được huấn luyện. Model đi qua dataset một lần (`1` epoch) với learning rate 10^{-4}. Mỗi chuỗi dài tối đa 1024 token.
- **Kết quả lưu ở đâu?** Log và checkpoint được ghi vào thư mục `output`.

Hai tham số về batch thì nên đọc cùng nhau. Mỗi GPU chỉ xử lý 1 mẫu mỗi lượt, nhưng gradient được cộng dồn qua 4 lượt rồi mới cập nhật tham số. Với 2 GPU, mỗi lần cập nhật thực chất dựa trên $1 \times 4 \times 2 = 8$ mẫu. Đây là cách thường dùng để có batch lớn hơn khi bộ nhớ GPU có hạn.

V. Luồng xử lý của `swift sft`

Vậy khi ta chạy câu lệnh trên, bên trong diễn ra những gì? Hình 6 chia quá trình này thành năm bước. Hình chỉ mô tả khái quát để nối các thành phần với nhau, không phải thứ tự chi tiết của từng hàm trong mã nguồn.



Hình 6: Luồng xử lý bên trong lệnh `swift sft`. Các tham số dòng lệnh (bên trái) điều khiển từng bước tương ứng.

Đầu tiên, framework đọc các tham số dòng lệnh (bước 1). Từ đó, model và tokenizer được nạp. Song song với việc này, dataset được định dạng theo chat template rồi chuyển thành token (bước 2).[2] Với LoRA, adapter được gắn vào model trong khi trọng số gốc giữ nguyên (bước 3). Hai

[2]Với model văn bản, mặc định MS-SWIFT duyệt qua toàn bộ dataset một lượt trước khi huấn luyện để kiểm tra và tính độ dài các mẫu. Trong lúc huấn luyện, từng mẫu được token hóa lại khi cần. Nếu dataset rất lớn,

nhánh này gặp nhau ở vòng lặp huấn luyện (bước 4). Tại đây, model học từ từng batch và chỉ tính loss trên phần trả lời của `assistant`. Cuối cùng, log và checkpoint được lưu vào thư mục đầu ra để dùng cho inference và evaluation (bước 5).

Trong hình, mỗi nhóm tham số của câu lệnh gắn với một công việc cụ thể. Nắm được mối liên hệ này, bạn sẽ dễ chọn cấu hình hơn. Khi áp dụng MS-SWIFT cho một dataset hoặc môi trường huấn luyện khác, bạn cũng biết cần thay đổi những gì.

VI. Thực hành fine-tune Qwen2.5 trên GSM8K với MS-SWIFT

Các phần trước đã giải thích từng thành phần và cách đọc một câu lệnh `swift sft`. Ở phần này, ta sẽ đi qua một notebook chạy thật từ đầu đến cuối. Notebook lấy dataset GSM8K từ Hugging Face rồi chuyển sang định dạng `messages`. Model được fine-tune là `Qwen/Qwen2.5-0.5B-Instruct`, dùng LoRA kết hợp DeepSpeed ZeRO-2 trên 2 GPU Tesla T4. Cuối cùng, ta chạy thử model và gộp adapter vào model gốc.

Notebook khá dài, nên bài đọc không chép lại toàn bộ code. Ta chỉ dừng lại ở những đoạn quyết định cách MS-SWIFT chạy, gồm cell cấu hình, hàm chuyển đổi dữ liệu và ba câu lệnh `swift sft`, `swift infer`, `swift export`. Những phần còn lại như in thông tin GPU hay in kết quả, bạn có thể xem trực tiếp trong notebook đi kèm.

VI.1. Cấu hình chung

Các giá trị có thể thay đổi được gom vào cell cấu hình như bên dưới. Notebook cũng sẽ tạo hai file dữ liệu (`gsm8k_train.jsonl` và `gsm8k_test.jsonl`) cùng với thư mục `output`.

```
MODEL_ID     = "Qwen/Qwen2.5-0.5B-Instruct"
HF_DATASET   = "openai/gsm8k"
HF_CONFIG    = "main"
NUM_TRAIN    = 512
NUM_TEST     = 8
DS_STAGE     = "zero2"
MAX_LENGTH   = 1024
TARGET_BATCH = 16
```

Phần cài đặt chỉ cần hai thư viện là MS-SWIFT và DeepSpeed.

```
pip install -U -q ms-swift deepspeed
pip uninstall -y torchao
```

Ở bước kiểm tra phần cứng, notebook chọn kiểu dữ liệu `bfloat16` hoặc `float16` cho model tuỳ theo GPU. Lựa chọn này sau đó được truyền vào lệnh huấn luyện ở Mục VI.3..[3]

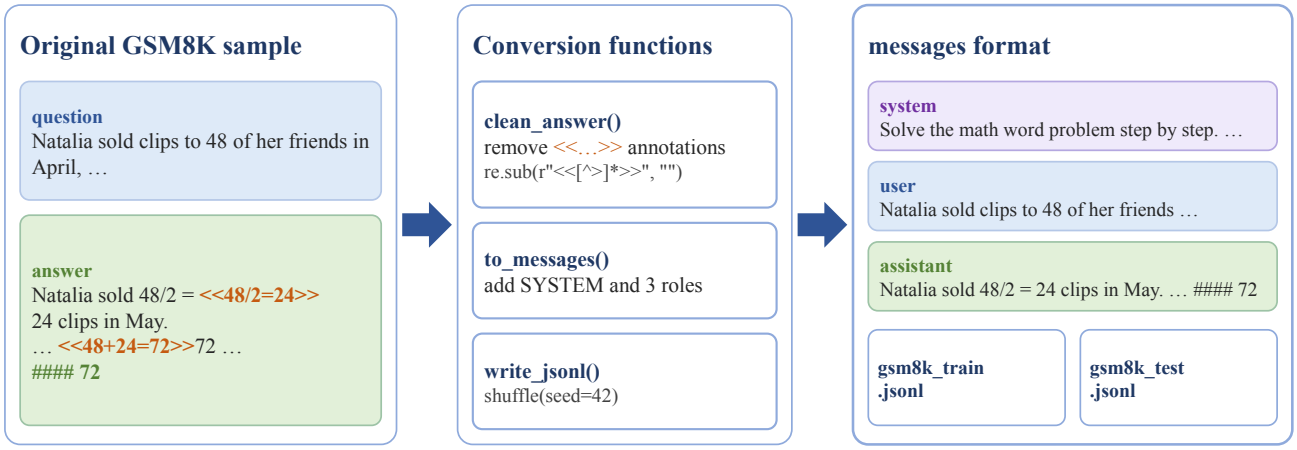
bạn có thể bật `-lazy_tokenize true` để bỏ qua lượt duyệt ban đầu. Bạn cũng có thể dùng `-streaming true` để đọc và xử lý dữ liệu dần trong lúc huấn luyện (khi đó cần đặt `-max_steps`). Cách còn lại là token hóa trước bằng `swift export -to_cached_dataset true`.

[3]Notebook dựa vào `torch.cuda.is_bf16_supported()` để chọn. Trên một số GPU cũ như T4 (compute

VI.2. Chuẩn bị dữ liệu GSM8K theo định dạng `messages`

GSM8K là tập bài toán đố bằng tiếng Anh, gồm 7473 mẫu train và 1319 mẫu test. Mỗi mẫu chỉ có hai trường. Trường `question` là đề bài, còn `answer` là lời giải từng bước, với đáp số nằm sau dấu `####`. Trong lời giải còn có những đoạn như `<<48/2=24>>`. Đây là chú thích phép tính mà GSM8K chèn vào. Model không cần học các đoạn này, nên ta bỏ chúng đi trước khi train.

Dữ liệu như vậy chưa có dạng hội thoại mà MS-SWIFT cần. Vì vậy, việc chuẩn bị chỉ là biến mỗi cặp `question`/`answer` thành một mẫu `messages` gồm ba lượt, như trong Hình 7.



Hình 7: Chuyển một mẫu GSM8K gốc sang định dạng `messages` của MS-SWIFT.

Ở bước chuyển đổi, bạn chỉ cần nắm hai hàm dưới đây. Hàm `clean_answer` dùng một biểu thức regex để xoá các chú thích `<<...>>`. Hàm `to_messages` đặt đề bài và lời giải vào đúng vai trò `user` và `assistant`.

```
SYSTEM = ("Solve the math word problem step by step. "
          "Give the final answer on the last line as '#### <number>'.")

def clean_answer(ans):
    return re.sub(r"<<[^>]*>>", "", ans).strip()

def to_messages(ex):
    return {"messages": [
        {"role": "system",    "content": SYSTEM},
        {"role": "user",      "content": ex["question"].strip()},
        {"role": "assistant", "content": clean_answer(ex["answer"])},
    ]}
```

Lượt `system` nhắc model giải từng bước và ghi đáp số ở dòng cuối theo dạng `#### <số>`, giống cách GSM8K trình bày lời giải. Sau đó, hàm `write_jsonl` xáo trộn mỗi split với `seed=42`. Hàm này lấy 512 mẫu từ split train và 8 câu từ split test, rồi ghi mỗi mẫu thành một dòng JSON. Model chưa thấy tám câu test này lúc train, nên ta sẽ dùng chúng để chạy thử ở cuối bài.

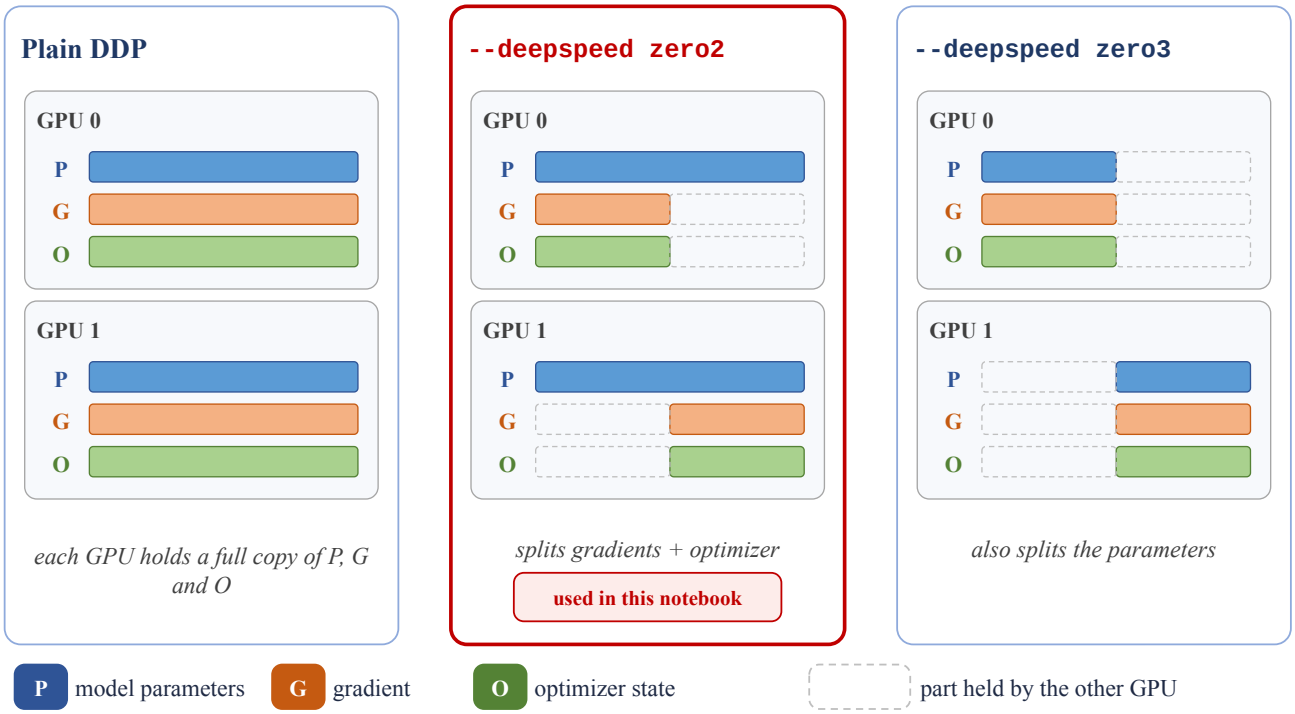
Trước khi train, bạn nên kiểm tra lại dữ liệu một lần. Notebook áp dụng chat template của

capability 7.5), hàm này vẫn có thể trả về `True` dù bf16 chỉ được giả lập và chạy chậm. Cách chắc chắn hơn là chỉ dùng bf16 khi `torch.cuda.get_device_capability()[0] >= 8`, còn lại dùng fp16.

Qwen2.5 lên một mẫu để xem chuỗi thực sự đi vào model, giống quy trình ở Hình 3. Sau đó nó đo độ dài token của cả 512 mẫu. Mẫu ngắn nhất có 91 token, mẫu dài nhất có 396 token. Tất cả đều nhỏ hơn `MAX_LENGTH = 1024`, nên không có mẫu nào bị loại bỏ. Muốn chắc rằng chính MS-SWIFT đọc được file, bạn có thể gọi thử `load_dataset` trong `swift.dataset`. Đến lúc train, ta chỉ cần trỏ `-dataset` vào file JSONL này.

VI.3. Huấn luyện mô hình bằng `swift sft` kết hợp LoRA và DeepSpeed ZeRO-2

Với model 0.5B tham số và LoRA, log huấn luyện chỉ ghi nhận khoảng 1.71 GiB bộ nhớ GPU, nghĩa là một GPU T4 đã đủ. Notebook vẫn bật DeepSpeed ZeRO để minh họa cách MS-SWIFT chia trạng thái huấn luyện khi có nhiều GPU. Hình 8 cho thấy mỗi GPU giữ những gì trong ba cách chạy với 2 GPU. Bạn có thể thử với các model lớn hơn để thấy rõ tác dụng của DeepSpeed ZeRO-2.



Hình 8: Cách chia tham số (P), gradient (G) và trạng thái optimizer (O) cho 2 GPU với DDP thường, `-deepspeed zero2` và `-deepspeed zero3`. Notebook dùng ZeRO-2.

Với DDP thường, GPU nào cũng giữ trọn ba phần. ZeRO-2 vẫn để mỗi GPU giữ đủ tham số nhưng chia đôi gradient và trạng thái optimizer. Nhờ đó, bộ nhớ dành cho hai thành phần này trên mỗi GPU giảm đi một nửa. ZeRO-3 đi xa hơn và chia luôn cả tham số. Hai biến thể `zero2_offload` và `zero3_offload` còn chuyển các phần này sang RAM của CPU để tiết kiệm VRAM, đổi lại tốc độ chậm hơn. Trong notebook, lựa chọn này nằm ở biến `DS_STAGE`.

```
NUM_GPUS = 2
gpu_ids = ",".join(str(i) for i in range(NUM_GPUS))   # "0,1"
PER_DEVICE_BS = 1
```

Với `TARGET_BATCH = 16`, ta được `GRAD_ACC = 8`, tức mỗi lần cập nhật dựa trên $1 \times 8 \times 2 = 16$ mẫu. Notebook cũng suy ra `gpu_ids` từ `NUM_GPUS`. Nhờ vậy, `CUDA_VISIBLE_DEVICES` luôn liệt kê số GPU đúng bằng `NPROC_PER_NODE`. Dưới đây là phần chính của lệnh huấn luyện, với các biến đã được thay bằng giá trị cụ thể.

```
CUDA_VISIBLE_DEVICES=0,1 NPROC_PER_NODE=2 \
swift sft \
    --model Qwen/Qwen2.5-0.5B-Instruct \
    --use_hf true \
    --dataset gsm8k_train.jsonl \
    --split_dataset_ratio 0.05 \
    --tuner_type lora \
    --lora_rank 8 \
    --lora_alpha 32 \
    --target_modules all-linear \
    --deepspeed zero2 \
    --torch_dtype bfloat16 --bf16 true --fp16 false \
    --num_train_epochs 1 \
    --per_device_train_batch_size 1 \
    --gradient_accumulation_steps 8 \
    --learning_rate 1e-4 \
    --warmup_ratio 0.05 \
    --max_length 1024 \
    --logging_steps 1 \
    --eval_steps 10 \
    --save_steps 10 \
    --save_total_limit 2 \
    --output_dir output
```

Phần lớn tham số đã quen thuộc từ câu lệnh mẫu ở phần trước. Các tham số mới cũng có thể đọc theo bốn nhóm.

- **Lấy model ở đâu, đánh giá trên dữ liệu nào?** `-use_hf true` yêu cầu tải model từ Hugging Face thay vì ModelScope như mặc định. `-split_dataset_ratio 0.05` tách 5% dữ liệu train ra làm tập validation để tính `eval_loss`.
- **LoRA được gắn vào đâu và lớn đến mức nào?** `-target_modules all-linear` gắn adapter vào mọi lớp linear của model. `-lora_rank 8` quyết định kích thước adapter, còn `-lora_alpha 32` điều chỉnh mức ảnh hưởng của adapter lên đầu ra của lớp gốc.
- **Chia bộ nhớ ra sao?** `-deepspeed zero2` bật ZeRO-2 như Hình 8. Ba tham số về kiểu dữ liệu được sinh ra từ lựa chọn bf16/fp16 ở bước kiểm tra phần cứng, nên chúng luôn khớp nhau.
- **Ghi log và lưu checkpoint khi nào?** Loss được ghi sau mỗi bước. Cứ 10 bước, model được đánh giá trên tập validation và lưu một checkpoint. `-save_total_limit 2` chỉ giữ lại checkpoint tốt nhất và checkpoint cuối cùng để tiết kiệm dung lượng.

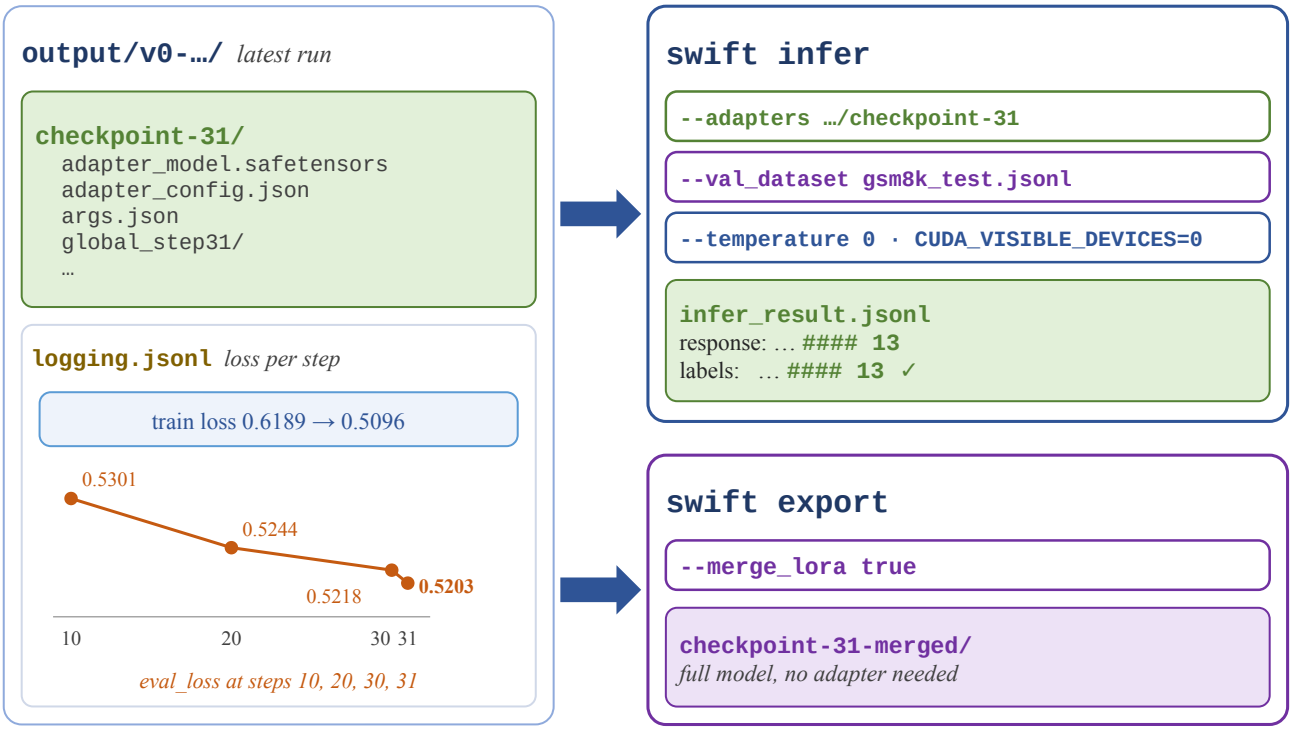
Lệnh trong notebook còn vài tham số phụ như `-attn_impl sdpa`, `-report_to none` hay `-dataloader_num_workers 0`. Chúng không làm thay đổi cách model học, nên ta lược bớt ở đây. Từ 512 mẫu, sau khi tách 5% làm validation và gom 16 mẫu cho mỗi lần cập nhật, ta có tổng cộng 31 bước huấn luyện. Cả quá trình hoàn tất trong khoảng 2 phút trên 2 GPU T4.

VI.4. Đọc kết quả

MS-SWIFT không ghi đè lần chạy cũ. Mỗi lần train tạo một thư mục con mới có dạng `output/vX-<thời gian>/`. Bên trong có file `logging.jsonl` ghi lại loss theo từng bước, cùng các thư mục `checkpoint-<step>/`.

```
RUN_DIR = max(Path(OUTPUT_DIR).glob("v*"), key=lambda p: p.stat().st_mtime)
CKPT = max(RUN_DIR.glob("checkpoint-*"),
           key=lambda p: int(p.name.split("-")[-1]))
recs = [json.loads(l) for l in open(RUN_DIR / "logging.jsonl") if l.strip()]
eval_loss = [(r["global_step/max_steps"], r["eval_loss"])
             for r in recs if "eval_loss" in r]
```

Trong lần chạy của notebook, checkpoint cuối là `checkpoint-31`. Thư mục này chứa adapter LoRA (`adapter_model.safetensors` và `adapter_config.json`). Bên cạnh đó là file `args.json` lưu lại toàn bộ tham số huấn luyện. Trạng thái của DeepSpeed nằm trong `global_step31/`. Phía bên trái Hình 9 tóm tắt cấu trúc này. Đọc `logging.jsonl`, ta thấy `eval_loss` giảm nhẹ qua các lần đánh giá, từ 0.5301 ở bước 10 xuống 0.5244 ở bước 20, 0.5218 ở bước 30 và 0.5203 ở bước 31.



Hình 9: Sau khi train, checkpoint và `logging.jsonl` trong thư mục của lần chạy mới nhất được dùng cho `swift infer` (chạy thử với adapter) và `swift export` (gộp LoRA vào model gốc).

Như đã nói ở phần log và checkpoint, loss giảm chưa đủ để kết luận model đã giải toán tốt hơn.

Vì thế, bước tiếp theo là cho model trả lời những câu nó chưa từng thấy.

VI.5. Chạy thử với `swift infer` và gộp LoRA bằng `swift export`

Để chạy thử, ta trỏ tham số `-adapters` vào checkpoint vừa tìm được, rồi đưa file test vào tham số `-val_dataset`.

```
env -u NPROC_PER_NODE CUDA_VISIBLE_DEVICES=0 \
swift infer \
    --adapters output/v0-.../checkpoint-31 \
    --val_dataset gsm8k_test.jsonl \
    --infer_backend transformers \
    --max_new_tokens 512 \
    --temperature 0 \
    --max_batch_size 4 \
    --result_path infer_result.jsonl
```

Lệnh này có vài điểm cần để ý. `swift infer` tự đọc `args.json` trong checkpoint, nên ta không cần khai lại `-model` hay system prompt. Inference chỉ cần một GPU. Vì vậy lệnh chọn GPU 0 và gỡ biến `NPROC_PER_NODE` để MS-SWIFT không khởi chạy nhiều tiến trình. Còn `-temperature 0` khiến model luôn chọn token có xác suất cao nhất, nên chạy lại bao nhiêu lần cũng cho cùng một câu trả lời.

Mỗi dòng trong `infer_result.jsonl` có câu trả lời của model ở trường `response` và lời giải tham khảo ở trường `labels`. Nhờ vậy, ta có thể đặt hai đáp số cạnh nhau để so sánh. Trong các câu được in ra, có câu model giải đúng, cùng đáp số `#### 13` với lời giải tham khảo. Nhưng cũng có câu model hiểu sai đề và ra 44 thay vì 89. Tám câu chỉ đủ để kiểm tra model đã học đúng định dạng trả lời. Muốn biết model giải toán tốt đến đâu, ta cần đánh giá trên nhiều câu hơn, chẳng hạn bằng `swift eval`.

Checkpoint ở trên chỉ chứa adapter LoRA, nên lúc nào cũng phải nạp kèm model gốc. Nếu cần một bộ trọng số hoàn chỉnh để triển khai, ta gộp adapter vào model bằng `swift export`.

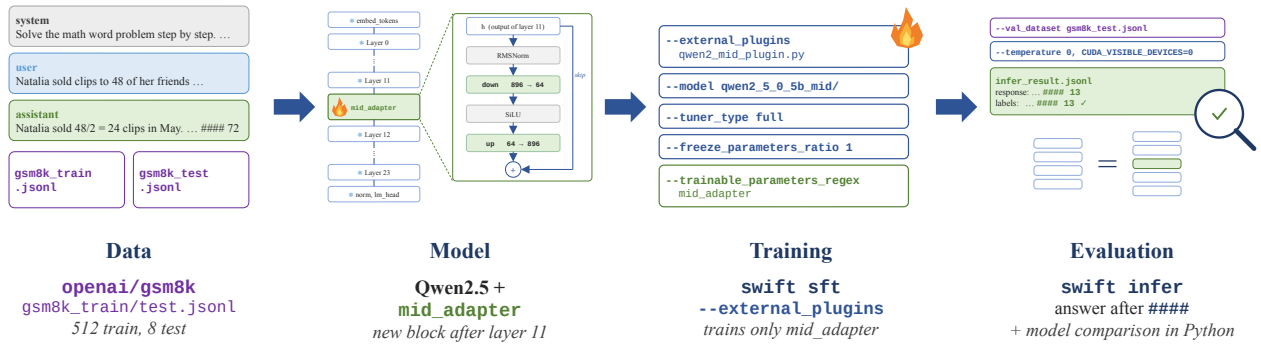
```
env -u NPROC_PER_NODE CUDA_VISIBLE_DEVICES=0 \
swift export \
    --adapters output/v0-.../checkpoint-31 \
    --merge_lora true
```

Model sau khi gộp nằm trong thư mục `checkpoint-31-merged`, ngay cạnh checkpoint gốc. Từ đây, ta có thể nạp model trực tiếp bằng `-model` mà không cần adapter nữa. Đổi lại, thư mục này chiếm thêm dung lượng đĩa bằng cả một model.[4]

[4]Lệnh `swift infer` và `swift export` trong notebook không truyền `-use_hf true`, nên log cho thấy model gốc được nạp từ ModelScope. Nếu muốn mọi bước đều tải từ Hugging Face, hãy thêm tham số này vào cả hai lệnh, hoặc đặt biến môi trường `USE_HF=1`.

VII. Thực hành chèn một khối mới vào giữa Qwen2.5 và huấn luyện bằng MS-SWIFT

Ở Mục VI., model được dùng nguyên trạng và chỉ có LoRA được gắn thêm vào. Trong thực tế, đôi khi ta muốn sửa chính kiến trúc model, chẳng hạn chèn thêm một khối mới vào giữa các decoder layer. Notebook thứ hai của bài đọc làm đúng việc này là giữ nguyên dữ liệu GSM8K và quy trình `swift sft` đến `swift infer`, nhưng thay model bằng một phiên bản Qwen2.5 có thêm khối `mid_adapter` giữa layer 11 và layer 12, rồi chỉ huấn luyện riêng khối này. Hình 10 tóm tắt bốn bước của notebook.



Hình 10: Tổng quan notebook thứ hai.

Phần dữ liệu và các lệnh `swift` đã được giải thích ở Mục VI., nên bài đọc chỉ dừng lại ở những chỗ mới: viết kiến trúc, lưu model thành một thư mục chuẩn Hugging Face, đăng ký model với MS-SWIFT, chọn đúng phần tham số để train và kiểm tra lại kết quả.

VII.1. Cấu hình và dữ liệu

Phần cài đặt giữ nguyên như notebook trước, kể cả bước gỡ `torchao`. Cell cấu hình cũng giống Mục VI., chỉ khác ở batch size hiệu dụng mong muốn, nay là 8, và có thêm hai thông số cho khối mới cùng thư mục lưu model đã sửa:

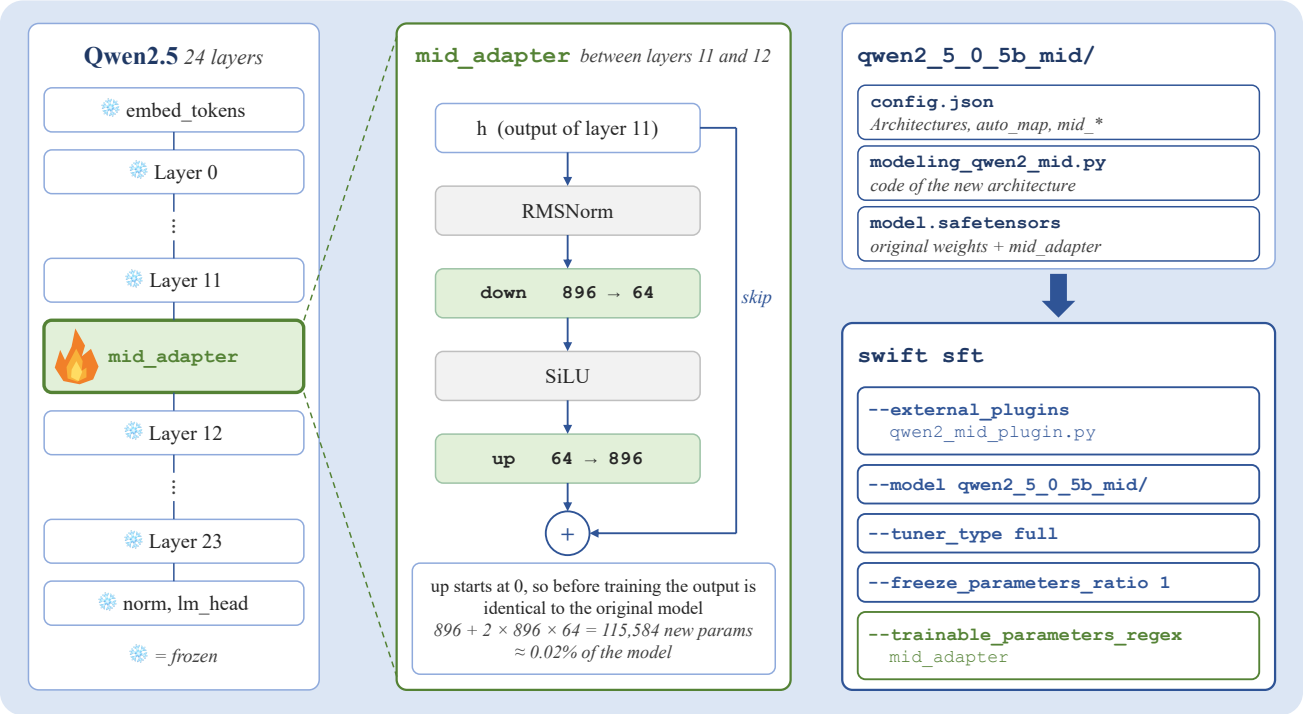
```
NUM_TRAIN    = 512
NUM_TEST     = 8
MAX_LENGTH   = 1024
TARGET_BATCH = 8
# Model moi: Qwen2.5 + mid_adapter
MID_LAYER_IDX   = 11    # chen sau layer 11 (tren tong so 24)
MID_ADAPTER_DIM = 64    # kich thuoc nut co chai cua adapter
MID_MODEL_DIR = WORK_DIR / "qwen2_5_0_5b_mid"
```

VII.2. Ý tưởng: khối `mid_adapter`

Khối mới được chèn vào giữa layer 11 và layer 12. Gọi h là đầu ra của layer 11, khối này tính

$$h \longmapsto h + \mathrm{up}\Big(\mathrm{SiLU}(\mathrm{down}(\mathrm{RMSNorm}(h)))\Big),$$

trong đó `down` nén vector 896 chiều xuống 64, còn `up` mở rộng vector lên lại 896. Kết quả được cộng vào h qua một đường tắt (residual connection), như phần giữa Hình 11. Điểm mấu chốt là `up` được khởi tạo bằng 0. Khi đó, ngay sau khi chèn, khối này cộng thêm đúng bằng 0 và model mới cho ra kết quả giống hệt model gốc. Ta không làm hỏng những gì model đã biết, chỉ mở thêm một chỗ để nó học.



Hình 11: Chèn `mid_adapter` vào giữa layer 11 và 12 của Qwen2.5-0.5B-Instruct. Bên trái minh họa các layer gốc bị đóng băng, chỉ khối mới được train. Giữa là cấu trúc của khối, với `up` khởi tạo bằng 0. Bên phải trình bày thư mục model mới và các tham số của lệnh `swift sft` chỉ train khối mới.

Để MS-SWIFT làm việc được với một kiến trúc tự viết, notebook đi theo ba bước. Thứ nhất, viết kiến trúc mới thành file `modeling_qwen2_mid.py`, kế thừa các lớp Qwen2 của `transformers`. Thứ hai, lưu model thành một thư mục chuẩn Hugging Face, trong đó `config.json` trỏ tới file code này. Thứ ba, đăng ký một `model_type` mới trong một plugin để MS-SWIFT biết model dùng chat template nào. Ba mục tiếp theo đi lần lượt qua từng bước.

VII.3. Một model Hugging Face gồm những gì?

Trước khi viết kiến trúc mới, ta cần biết thư viện `transformers` lưu và nạp một model ra sao. Phần sửa kiến trúc ở các mục sau đều dựa vào cơ chế này.

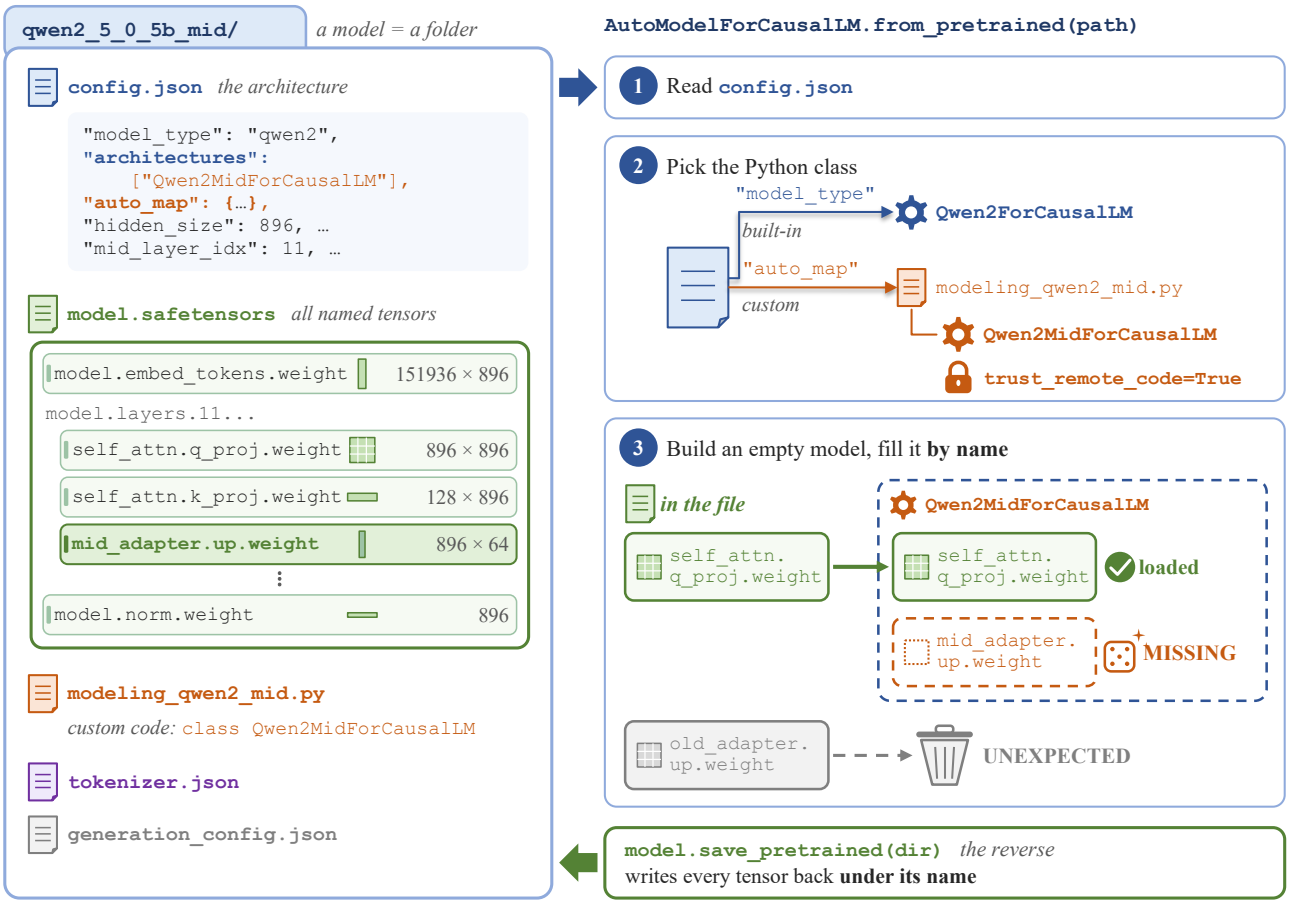
Một model là một thư mục. Khi tải `Qwen/Qwen2.5-0.5B-Instruct` về, ta không nhận được một file duy nhất mà là cả một thư mục. Trong đó có:

- `config.json` mô tả kiến trúc, chẳng hạn số layer (`num_hidden_layers`), kích thước vector ẩn (`hidden_size`), và quan trọng nhất là trường `architectures`, cho biết lớp Python nào sẽ dựng nên model.
- `model.safetensors` chứa trọng số. Có thể hình dung file này như một bảng tra, mỗi dòng gồm *tên* của một tham số, ví dụ `model.layers.11.self_attn.q_proj.weight`, và *tensor* tương ứng. Model lớn thì được chia thành nhiều file `model-0000x-of-0000y.safetensors`, kèm một file chỉ mục.
- Các file tokenizer (`tokenizer.json`, `tokenizer_config.json`, ...). Chat template ở Mục III.2 cũng được lưu trong các file này.
- `generation_config.json` các giá trị mặc định khi sinh văn bản, như `temperature` hay token kết thúc.

Nạp model `from_pretrained`.
Khi gọi `AutoModelForCausalLM.from_pretrained(path)`, `transformers` làm ba việc, minh họa trong Hình 12.

1. Đọc `config.json`.
2. Chọn lớp Python để dựng model. Với các kiến trúc có sẵn như Qwen2, thư viện tra `model_type` hoặc `architectures` để tìm ra lớp `Qwen2ForCausalLM`. Với kiến trúc tự viết, thư viện không biết lớp đó nằm ở đâu. Vì vậy `config.json` phải có thêm trường `auto_map` trỏ tới file code đặt cùng thư mục, và ta phải truyền `trust_remote_code=True` để cho phép chạy file code đó.
3. Dựng một model chưa được nạp trọng số theo config, rồi tải trọng số từ `model.safetensors` vào. Việc ghép này dựa hoàn toàn vào tên tham số. Tensor nào có tên trùng với một tham số trong model thì được nạp vào đúng chỗ đó. Tên nào model có mà file không có thì bị báo `MISSING` và được khởi tạo ngẫu nhiên. Tên nào file có mà model không dùng thì bị báo `UNEXPECTED` và bỏ qua.

Lưu model `save_pretrained`.
Hàm này làm ngược lại bằng cách ghi cấu hình hiện tại ra `config.json` và toàn bộ trọng số ra `model.safetensors`, mỗi tensor kèm tên của nó. Thư mục lưu ra có cấu trúc giống hệt thư mục tải về, nên có thể nạp lại bằng `from_pretrained` như bất kỳ model nào khác. Checkpoint LoRA ở Mục VI.4 cũng theo cùng ý tưởng, chỉ khác là `adapter_config.json` và `adapter_model.safetensors` chỉ chứa phần adapter, nên luôn phải nạp kèm model gốc.



Hình 12: Cấu trúc một thư mục model Hugging Face và cách dựng model rồi ghép trọng số theo tên tham số.

Từ cơ chế này, ta rút ra hai điều cho các mục tiếp theo. Thứ nhất, nếu kiến trúc mới giữ nguyên tên của mọi tham số cũ, toàn bộ trọng số Qwen2.5 gốc sẽ tự nạp vào đúng chỗ. Chỉ các tham số của khối mới bị báo `MISSING`, và đó là điều ta mong đợi. Thứ hai, để người khác (hay MS-SWIFT) nạp được model tự viết, thư mục model phải có đủ ba thứ: file code, trường `architectures` và `auto_map` trong `config.json`, cùng với cờ `trust_remote_code=True` lúc nạp.

VII.4. Viết kiến trúc mới

Đầu tiên, chúng ta ghi file `modeling_qwen2_mid.py` vào thư mục làm việc. Khối adapter là một `nn.Module` nhỏ, với hàm khởi tạo đặt `up` về 0 như sau:

```
class MidAdapter(nn.Module):
    def __init__(self, hidden_size, adapter_dim, eps=1e-6):
        super().__init__()
        self.norm = Qwen2RMSNorm(hidden_size, eps=eps)
        self.down = nn.Linear(hidden_size, adapter_dim, bias=False)
        self.act = nn.SiLU()
        self.up = nn.Linear(adapter_dim, hidden_size, bias=False)
        self.reset_parameters()

    @torch.no_grad()
```

```
    def reset_parameters(self):
        self.norm.weight.fill_(1.0)
        nn.init.normal_(self.down.weight, std=0.02)
        nn.init.zeros_(self.up.weight)   # zero-init

    def forward(self, hidden_states):
        return hidden_states + self.up(
            self.act(self.down(self.norm(hidden_states))))
```

Khối này không đứng thành một layer riêng trong danh sách `model.layers`. Thay vào đó, layer 11 được thay bằng một lớp con của `Qwen2DecoderLayer`, gọi `forward` gốc rồi đưa kết quả qua adapter:

```
class Qwen2MidDecoderLayer(Qwen2DecoderLayer):
    def __init__(self, config, layer_idx):
        super().__init__(config, layer_idx)
        self.mid_adapter = MidAdapter(config.hidden_size,
                                      get_mid_adapter_dim(config),
                                      config.rms_norm_eps)

    def forward(self, *args, **kwargs):
        outputs = super().forward(*args, **kwargs)
        if isinstance(outputs, tuple):     # transformers ban cu
            return ((self.mid_adapter(outputs[0]),)
                    + tuple(outputs[1:]))
        return self.mid_adapter(outputs)

class Qwen2MidForCausalLM(Qwen2ForCausalLM):
    def __init__(self, config):
        super().__init__(config)
        k = get_mid_layer_idx(config)
        self.model.layers[k] = Qwen2MidDecoderLayer(config, k)
        self.post_init()
```

Cách làm này có hai lợi ích. Tên mọi tham số cũ vẫn giữ nguyên, chẳng hạn `model.layers.11.self_attn...`, nên trọng số Qwen2.5 gốc nạp vào đúng chỗ, và model chỉ có thêm các tham số `model.layers.11.mid_adapter.*`. Ngoài ra, nhánh `isinstance` giúp lớp mới chạy được với cả phiên bản `transformers` trả về tensor lẫn phiên bản trả về tuple. Notebook cũng không tạo lớp config riêng: hai thông số `mid_layer_idx` và `mid_adapter_dim` chỉ là hai trường thêm vào `Qwen2Config`, đọc bằng `getattr` trong `get_mid_layer_idx` và `get_mid_adapter_dim`. Cuối file còn có hàm `reset_mid_adapter()` gọi lại `reset_parameters()` của khối mới.

VII.5. Dựng model mới và kiểm tra trước khi train

Sau khi đã định nghĩa mô hình, tiếp theo chúng ta đọc config của Qwen2.5, thêm hai trường mới, đổi `architectures` và thêm `auto_map` để `AutoModelForCausalLM` biết lấy lớp nào từ file code. Sau đó notebook nạp trọng số gốc vào lớp mới và lưu tất cả vào `MID_MODEL_DIR`:

```
config = AutoConfig.from_pretrained(MODEL_ID)
config.mid_layer_idx = MID_LAYER_IDX
config.mid_adapter_dim = MID_ADAPTER_DIM
config.architectures = ["Qwen2MidForCausalLM"]
config.auto_map = {"AutoModelForCausalLM":
```

```
                    "modeling_qwen2_mid.Qwen2MidForCausalLM"}

mid_model = Qwen2MidForCausalLM.from_pretrained(
    MODEL_ID, config=config, torch_dtype=torch.bfloat16)
mid_model.reset_mid_adapter()
mid_model.save_pretrained(MID_MODEL_DIR)
AutoTokenizer.from_pretrained(MODEL_ID).save_pretrained(
    MID_MODEL_DIR)
shutil.copy(WORK_DIR / "modeling_qwen2_mid.py", MID_MODEL_DIR)
```

Khi nạp, `transformers` báo ba tham số `model.layers.11.mid_adapter.{up, norm, down}.weight` là MISSING. Đây là điều đúng, vì chúng chưa từng tồn tại trong checkpoint gốc. Cũng vì thế mà `transformers` tự khởi tạo ngẫu nhiên ba tham số này, nên notebook phải gọi `reset_mid_adapter()` ngay sau đó để đưa `up` về 0. Output cho thấy khối mới có 115.584 tham số trên tổng số 494.148.352, tức khoảng 0,023%. Con số này khớp với phép tính trong Hình 11: $896 + 2 \times 896 \times 64 = 115.584$, gồm trọng số của `norm`, `down` và `up`. Thư mục `qwen2_5_0_5b_mid` sau khi lưu chứa `config.json`, `model.safetensors`, `modeling_qwen2_mid.py`, cùng các file tokenizer, chat template và `generation_config.json`.

Trước khi train, notebook kiểm tra lại rằng model mới thật sự hành xử giống model gốc. Model gốc và model mới được nạp lần lượt, trong đó model mới được nạp từ thư mục vừa lưu với `trust_remote_code=True`, giống hệt cách MS-SWIFT sẽ nạp. Cả hai nhận cùng một câu hỏi ("Thủ đô của Việt Nam là gì?"), rồi notebook so sánh số tham số, logits và câu trả lời sinh ra:

```
for name, path, remote in [("goc", MODEL_ID, False),
                           ("moi", MID_MODEL_DIR, True)]:
    m = AutoModelForCausalLM.from_pretrained(
        path, trust_remote_code=remote,
        torch_dtype=torch.float32).to(device).eval()
    ...
diff = (outs["goc"]["logits"]
        - outs["moi"]["logits"]).abs().max().item()
assert outs["goc"]["text"] == outs["moi"]["text"]
assert diff < 1e-4
```

Có một lưu ý là phép so sánh được làm ở `float32`, vì ở `bfloat16`, chạy lại cùng một model hai lần cũng có thể cho logits lệch nhau đôi chút do sai số làm tròn. Kết quả trong output khớp với mong đợi: model gốc (`Qwen2ForCausalLM`) có 494.032.768 tham số, model mới (`Qwen2MidForCausalLM`) có 494.148.352 tham số, chênh nhau đúng 115.584. Sai khác logits lớn nhất là `0.000e+00`, và hai câu trả lời giống nhau từng chữ. Đây là bước nên làm mỗi khi sửa kiến trúc, vì nó bắt được ngay những lỗi như quên đưa `up` về 0 hay nạp nhầm trọng số.

VII.6. Đăng ký `model_type` mới cho MS-SWIFT

Thư mục model đã đủ để `transformers` dựng lại kiến trúc mới, nhưng MS-SWIFT còn cần biết model này dùng chat template nào. Notebook khai báo điều này trong file `qwen2_mid_plugin.py`:

```
from swift.model import ModelMeta, register_model

register_model(
    ModelMeta(
        model_type="qwen2_mid",
```

```
        model_groups=[],            # model local, khong co id hub
        template="qwen2_5",         # chat template cua Qwen2.5
        architectures=["Qwen2MidForCausalLM"],
        additional_saved_files=["modeling_qwen2_mid.py"],
    ))
```

Trường `architectures` khớp với trường cùng tên trong `config.json`, nên khi ta trỏ `-model` vào thư mục model mới, MS-SWIFT tự nhận ra `model_type` mà không cần truyền `-model_type`. `template="qwen2_5"` dùng lại chat template của Qwen2.5, vì tokenizer và cách hội thoại không đổi. `additional_saved_files` nhắc MS-SWIFT chép file code sang mỗi checkpoint, để checkpoint cũng nạp lại được như một model độc lập. Bộ nạp mặc định của MS-SWIFT đã gọi `from_pretrained` với `trust_remote_code=True`, nên ta không cần viết thêm hàm nạp. Mọi lệnh `swift sft` hay `swift infer` dùng model này đều phải kèm `-external_plugins` trỏ tới file plugin.

VII.7. Huấn luyện chỉ riêng khối mới

Cách tính `GRAD_ACC` giữ nguyên như Mục VI.3.. Với `NUM_GPUS = 2` và `PER_DEVICE_BS = 4`, ta được `GRAD_ACC = 1`, tức mỗi lần cập nhật vẫn dựa trên $4 \times 1 \times 2 = 8$ mẫu. Dưới đây là phần chính của lệnh huấn luyện, với các biến đã được thay bằng giá trị trong lần chạy:

```
CUDA_VISIBLE_DEVICES=0,1 NPROC_PER_NODE=2 \
swift sft \
    --external_plugins qwen2_mid_plugin.py \
    --model qwen2_5_0_5b_mid \
    --use_hf true \
    --dataset gsm8k_train.jsonl \
    --split_dataset_ratio 0.05 \
    --tuner_type full \
    --freeze_parameters_ratio 1 \
    --trainable_parameters_regex mid_adapter \
    --torch_dtype bfloat16 --bf16 true --fp16 false \
    --num_train_epochs 3 \
    --per_device_train_batch_size 4 \
    --gradient_accumulation_steps 1 \
    --learning_rate 1e-3 \
    --warmup_ratio 0.05 \
    --max_length 1024 \
    --eval_steps 50 \
    --save_steps 50 \
    --save_total_limit 2 \
    --save_only_model true \
    --output_dir output
```

So với lệnh LoRA ở Mục VI.3., lệnh này khác ở bốn nhóm tham số:

- **Model nào?** `-external_plugins` nạp plugin vừa viết, còn `-model` là thư mục model mới thay vì một model id trên hub.

- **Train phần nào?** `-tuner_type full` cho MS-SWIFT làm việc trực tiếp trên tham số của model, không bọc thêm LoRA. `-freeze_parameters_ratio 1` đóng băng toàn bộ model, rồi `-trainable_parameters_regex mid_adapter` mở khoá lại đúng những tham số có tên chứa `mid_adapter`.
- **Học nhanh đến đâu?** Vì chỉ khoảng 0,1 triệu tham số được cập nhật, learning rate được đặt cao hơn LoRA (`1e-3` so với `1e-4`) và số epoch tăng lên 3.
- **Lưu gì?** Ở chế độ full, mỗi checkpoint là một model hoàn chỉnh, khoảng 1 GB. `-save_only_model true` bỏ qua trạng thái optimizer để tiết kiệm ổ đĩa. Lệnh này cũng không dùng DeepSpeed, vì phần được train quá nhỏ để cần chia bộ nhớ.

Sau khi fine-tune xong, chúng ta thấy các giá trị `eval_loss` gần như đứng yên: 0.5237 ở bước 50, 0.5284 ở bước 100, 0.5276 ở bước 150 và 0.5254 ở bước 183. MS-SWIFT vì vậy chọn `checkpoint-50` làm `best_model_checkpoint`, trong khi cell đọc kết quả vẫn lấy checkpoint cuối. Với một khối chỉ chiếm 0,02% model, kết quả này không có gì bất ngờ. Mục tiêu của notebook là cho thấy cả quy trình chạy đúng với một kiến trúc tự viết, chứ không phải làm model giải toán giỏi hơn.

VII.8. Kiểm tra khối mới và suy luận của mô hình

Để chắc chắn việc đóng băng và mở khoá hoạt động đúng, notebook so sánh từng tensor trong `model.safetensors` của thư mục model trước khi train với checkpoint sau khi train:

```
before = load_file(MID_MODEL_DIR / "model.safetensors")
after  = load_file(CKPT / "model.safetensors")
changed = [k for k in before
           if not torch.equal(before[k].float(), after[k].float())]
assert changed and all("mid_adapter" in k for k in changed)
```

Output cho thấy 2 trên 293 tensor thay đổi: `model.layers.11.mid_adapter.down.weight` và `model.layers.11.mid_adapter.up.weight`. Mọi tensor của model gốc giữ nguyên tuyệt đối, đúng như mong đợi. Cuối cùng, vì checkpoint ở chế độ full là một thư mục model hoàn chỉnh, ta đưa thẳng nó vào `-model` thay cho `-adapters` như ở Mục VI., và vẫn giữ `-external_plugins` để MS-SWIFT biết `qwen2_mid` là gì:

```
env -u NPROC_PER_NODE CUDA_VISIBLE_DEVICES=0 \
swift infer \
    --external_plugins qwen2_mid_plugin.py \
    --model output/v.../checkpoint-183 \
    --val_dataset gsm8k_test.jsonl \
    --infer_backend transformers \
    --max_new_tokens 512 \
    --temperature 0 \
    --max_batch_size 4 \
    --result_path infer_result.jsonl
```

Các tham số còn lại giống hệt lệnh `swift infer` ở Mục VI.. Log cho thấy MS-SWIFT đọc `args.json` trong checkpoint, tự đặt `model_type='qwen2_mid'` và `template='qwen2_5'`, rồi dựng lại đúng `Qwen2MidForCausalLM` với khối `mid_adapter` trong layer 11. Ở ba câu được in ra, model giải đúng câu về số chữ cái trong tên (`#### 13`), nhưng sai hai câu còn lại: ra 24 thay vì 109 và 104 thay vì 89. Cũng như ở Mục VI., tám câu chỉ đủ để kiểm tra model mới nạp và sinh đúng định dạng, chưa đủ để đánh giá khả năng giải toán.

Thông qua cả hai notebook, chúng ta đã biết được cách để dùng một kiến trúc tự viết với MS-SWIFT. Ta cần ba thứ: một file modeling chuẩn `transformers`, một thư mục model có `auto_map` trỏ tới file đó, và một plugin gọi `register_model`. Sau đó model được train như mọi model khác. Việc chọn phần nào được train chỉ nằm ở các tham số `-tuner_type`, `-freeze_parameters_ratio` và `-trainable_parameters_regex`. Để thực hành, các bạn có thể làm hai hướng mở rộng. Hướng thứ nhất là kết hợp LoRA với khối mới qua `-modules_to_save mid_adapter`; khi đó trọng số của khối mới được lưu luôn trong checkpoint LoRA, và lúc suy luận ta dùng `-adapters` kèm `-external_plugins`. Hướng thứ hai là biến model thành một model phân loại bằng `-task_type seq_cls` cùng `-num_labels`, để MS-SWIFT thay `lm_head` bằng một đầu phân loại.

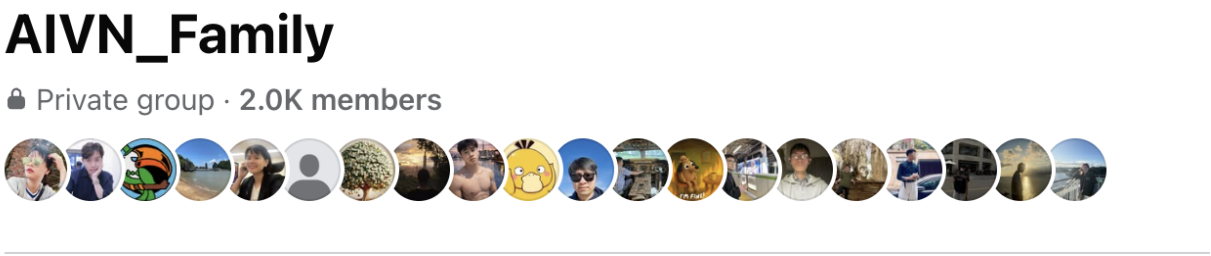
Phụ lục

1. **Tài nguyên thực hành**

   Notebook, code minh họa và các tài nguyên đi kèm bài đọc được cung cấp tại thư mục tài nguyên. Bạn có thể tham khảo notebook để chạy lại hai thực nghiệm fine-tuning Qwen2.5 trên GSM8K và huấn luyện mô hình có thêm khối `mid_adapter`.

2. **Hỏi đáp**

   Bạn có thể đặt câu hỏi về nội dung bài đọc tại nhóm Facebook hỏi đáp AIO. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.



Hình 13: Nhóm hỏi đáp AIO trên Facebook.

3. **Tiêu chí đánh giá**

Phần	Kiến thức	Yêu cầu đạt được
1	Các thành phần trong workflow MS-SWIFT: model, tokenizer, dataset, chat template, LoRA, full fine-tuning, batch size hiệu dụng, GPU, log và checkpoint.	Giải thích vai trò của từng thành phần, đọc được một lệnh `swift sft` và tính batch size hiệu dụng từ cấu hình huấn luyện.
2	Chuẩn bị GSM8K theo định dạng `messages`; fine-tuning Qwen2.5 bằng LoRA và DeepSpeed ZeRO-2; validation, inference và gộp adapter.	Chuyển dữ liệu sang JSONL, xác định các tham số huấn luyện, đọc log và checkpoint, chạy `swift infer`, dùng `swift export` và nhận biết giới hạn của việc đánh giá trên ít mẫu.
3	Khối `mid_adapter`, residual connection, zero initialization, cấu trúc model Hugging Face, `auto_map`, plugin MS-SWIFT và lựa chọn tham số cần huấn luyện.	Giải thích cách chèn khối mới vào Qwen2.5, lưu và đăng ký kiến trúc tùy chỉnh, chỉ mở khóa tham số của `mid_adapter`, kiểm tra trọng số gốc không đổi và chạy inference từ checkpoint.